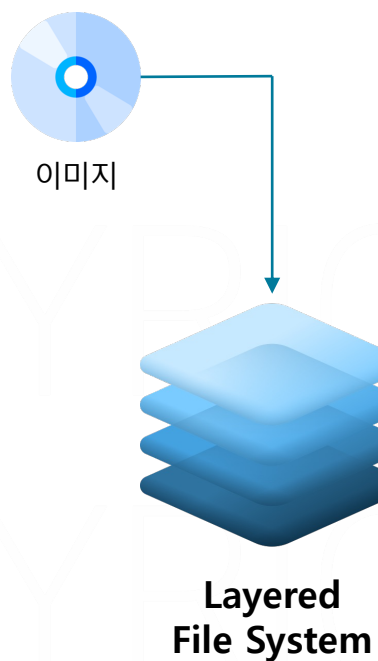


쉬운 도커

PART4. 이미지 빌드

PART4. 이미지 빌드

이미지와 레이어



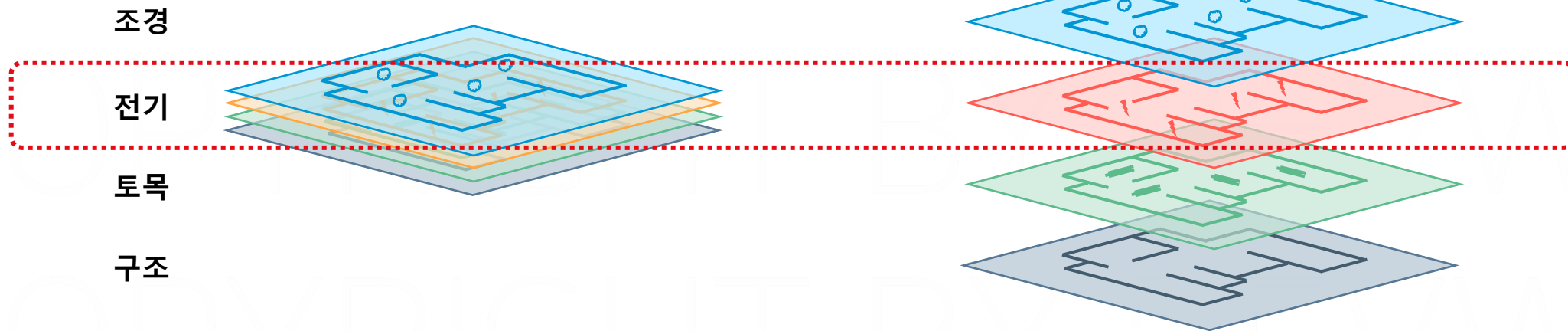
```
🍏 ~/ docker run -p 80:80 nginx
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
4ee097f9a366: Pull complete
6710b2157bb5: Pull complete
76d048093f36: Pull complete
658197f4b592: Pull complete
a2543a59b279: Pull complete
3972a57e5575: Pull complete
82359da50743: Pull complete
```



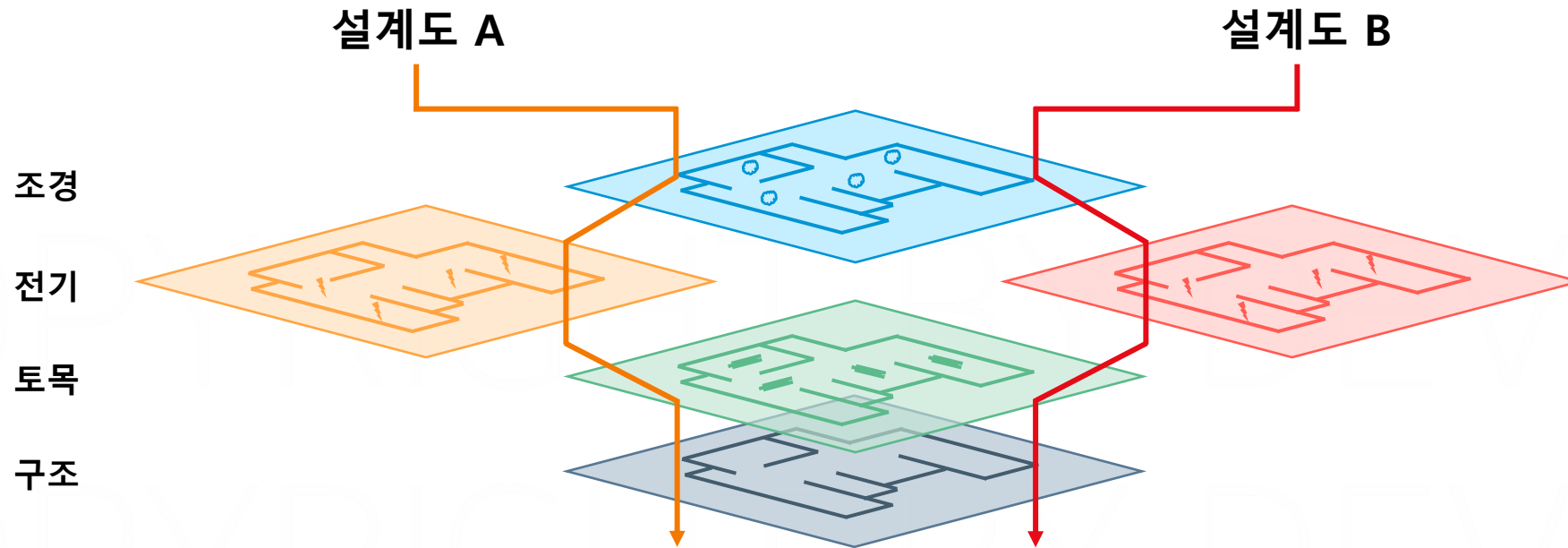
- 이미지의 레이어 구조는 재활용에 유리

설계도 A

설계도 B



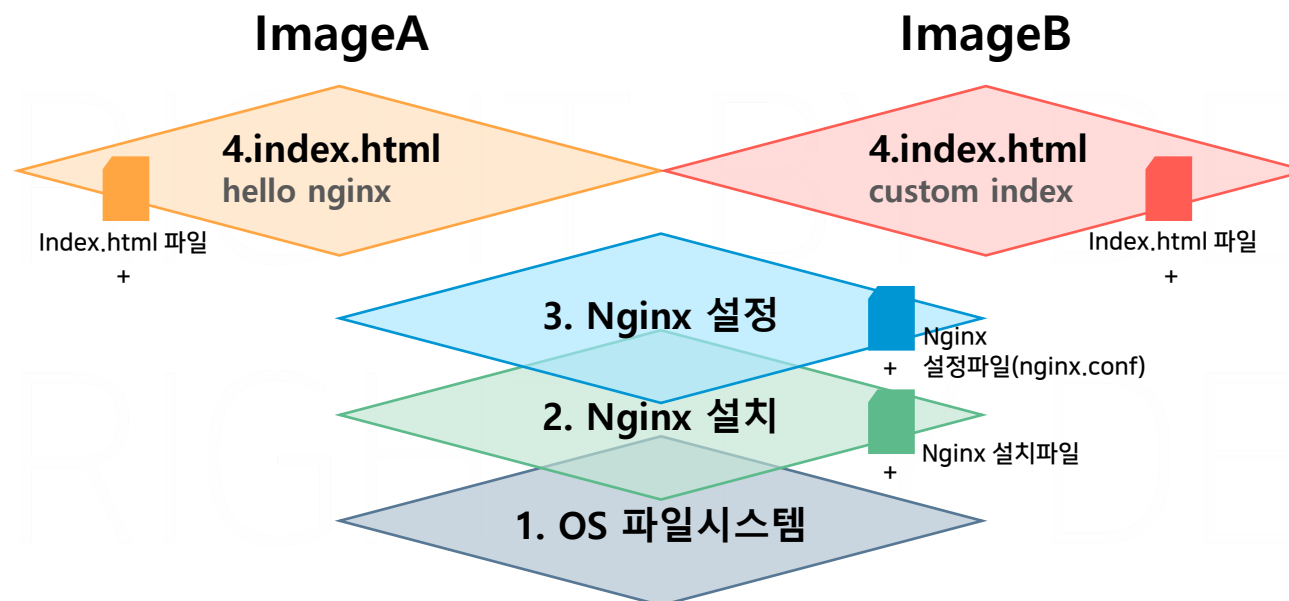
- 이미지의 레이어 구조는 재활용에 유리
- 효율적인 데이터 저장과 전송



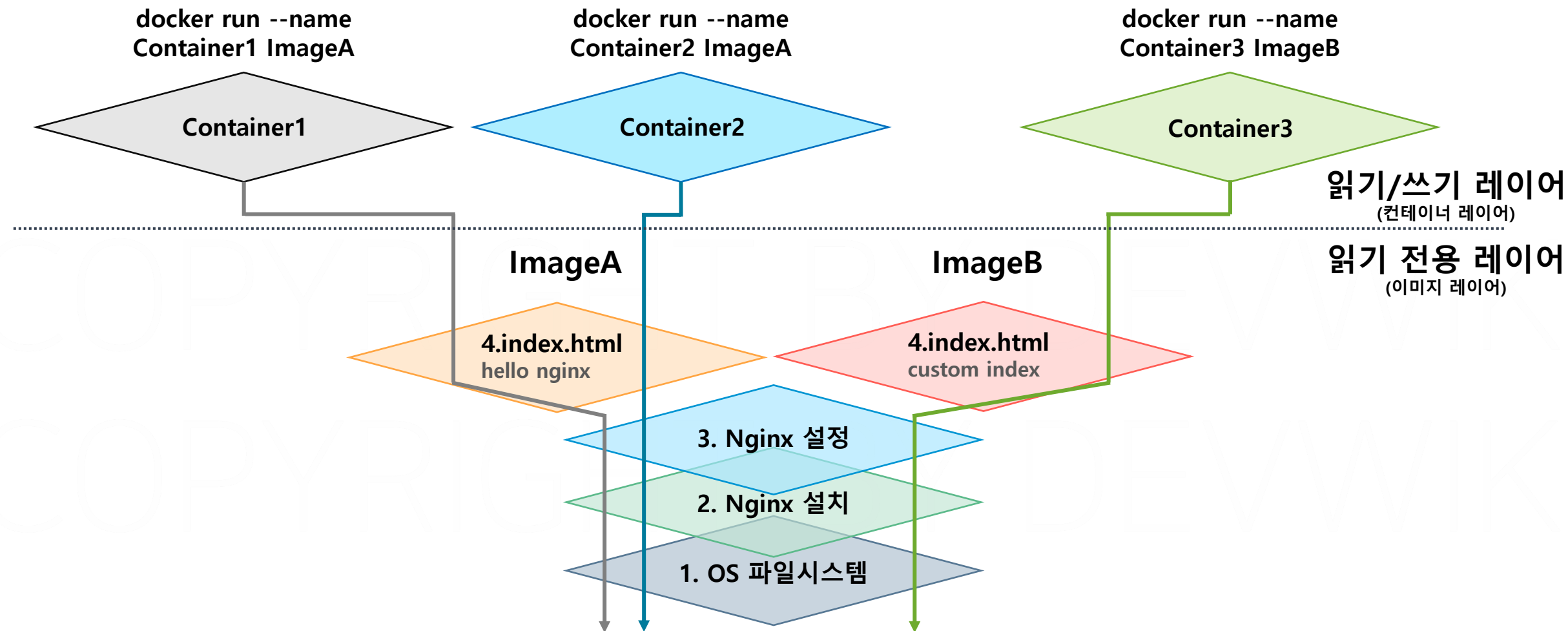
- 이미지의 레이어는 이전 레이어의 변경 사항이 저장

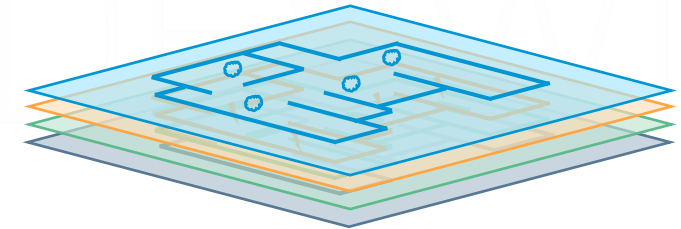
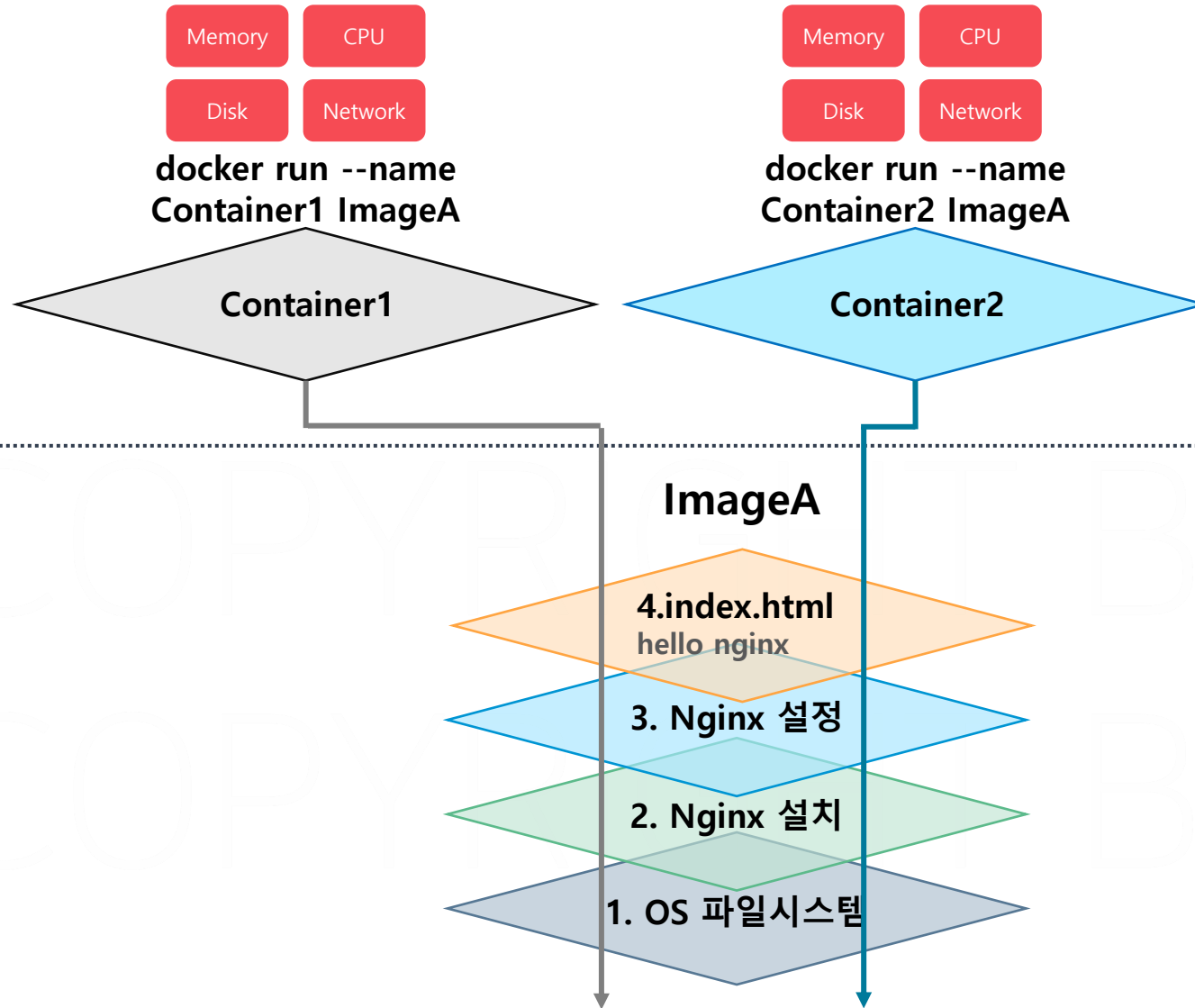
Nginx 이미지 설계

OS ► Nginx 설치 ► Nginx 설정 ► Index.html 파일 수정



- 이미지의 레이어는 이전 레이어의 변경 사항이 저장
- 컨테이너 실행 시 읽기, 쓰기 가능한 새로운 레이어 추가





빌딩 설계도

docker image history 이미지명

이미지의 레이어 이력 조회

실습 명령어

1. 실습용 이미지 다운로드

```
docker image pull devwikirepo/hello-nginx
```

```
docker image pull devwikirepo/config-nginx
```

```
docker image pull devwikirepo/pre-config-nginx
```

2. 실습용 이미지 히스토리 조회

```
docker image history devwikirepo/hello-nginx
```

```
docker image history devwikirepo/config-nginx
```

```
docker image history devwikirepo/pre-config-nginx
```

3. 실습용 이미지 레이어 조회

```
docker image inspect devwikirepo/hello-nginx
```

```
docker image inspect devwikirepo/config-nginx
```

```
docker image inspect devwikirepo/pre-config-nginx
```

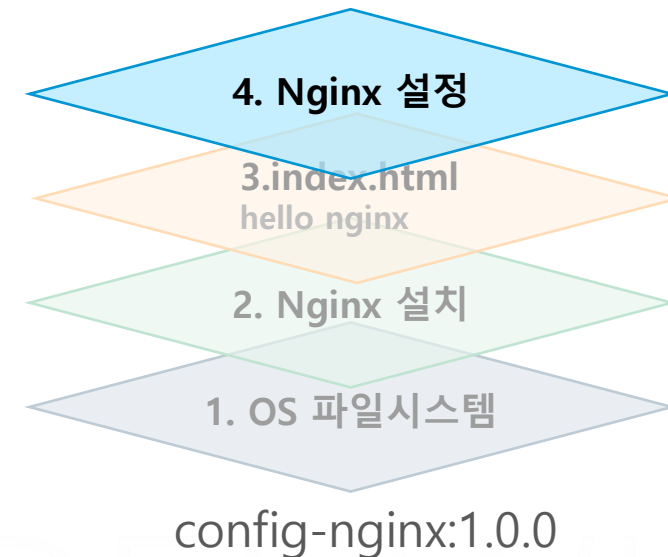
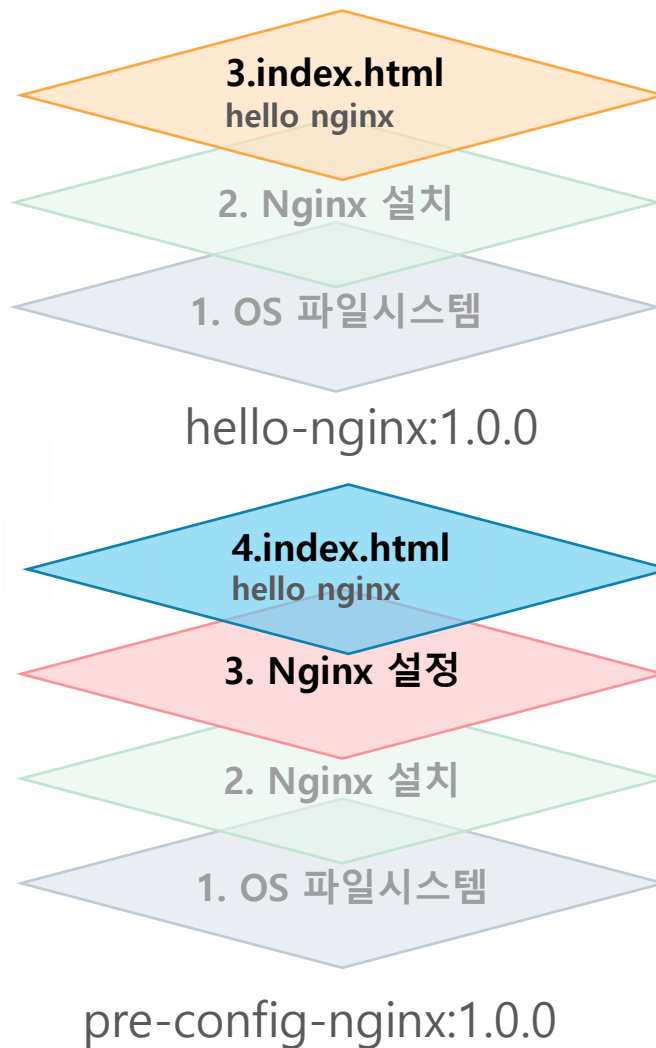
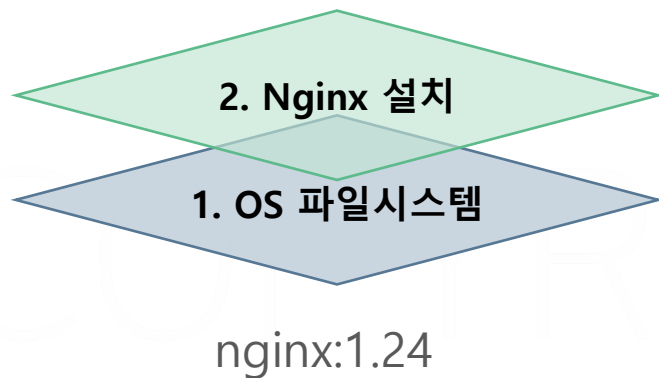
nginx:1.24.0

hello-nginx

config-nginx

pre-config-nginx

[illegible]



Layering : 각 레이어는 이전 레이어 위에 쌓이며, 여러 이미지 간에 공유될 수 있습니다. 레이어 방식은 **중복 데이터를 최소화**하고, **빌드 속도**를 높이며, **저장소를 효율적으로 사용할** 수 있게 해줍니다.

Copy-on-Write (CoW) 전략 : 다음 레이어에서 이전 레이어의 특정 파일을 수정 할 때, **해당 파일의 복사본을 만들어서 변경 사항을 적용**합니다. 이렇게 함으로써 원래 레이어는 수정되지 않고 그대로 유지됩니다.

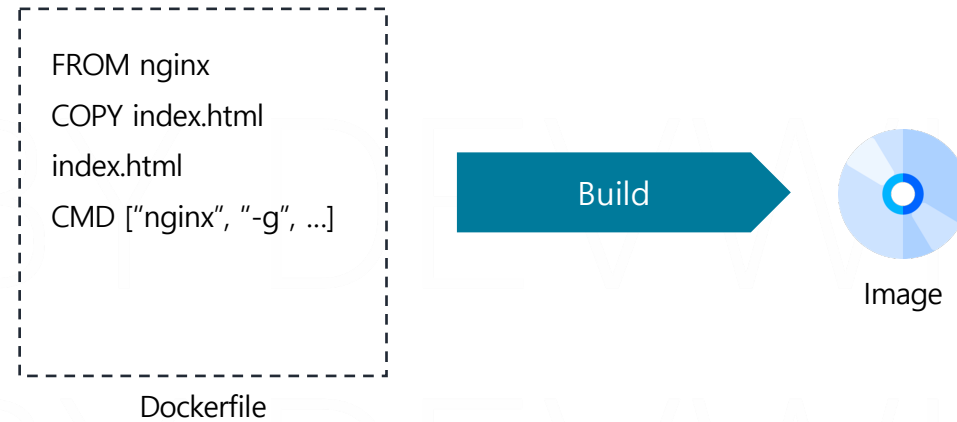
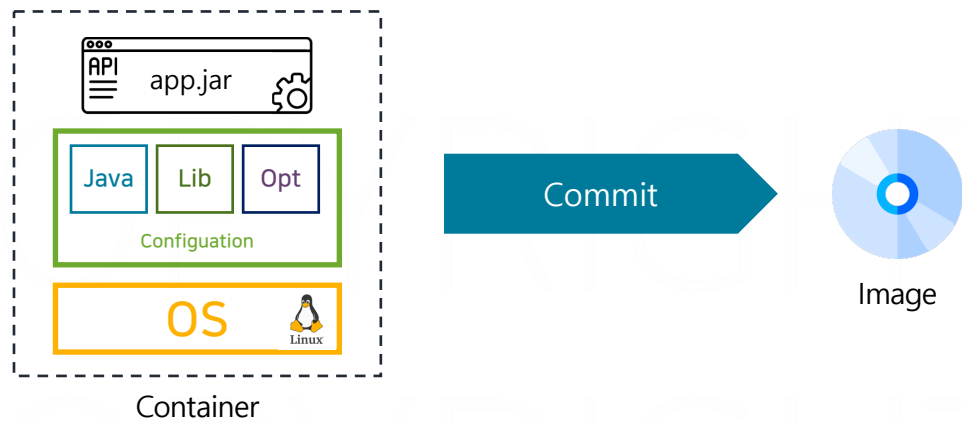
Immutable Layers (불변 레이어) : 이미지의 각 레이어는 불변으로, 한 번 생성되면 변경되지 않습니다. 이렇게 함으로써 **이미지의 일관성을 유지**하고, 여러 컨테이너에서 안전하게 공유할 수 있습니다.

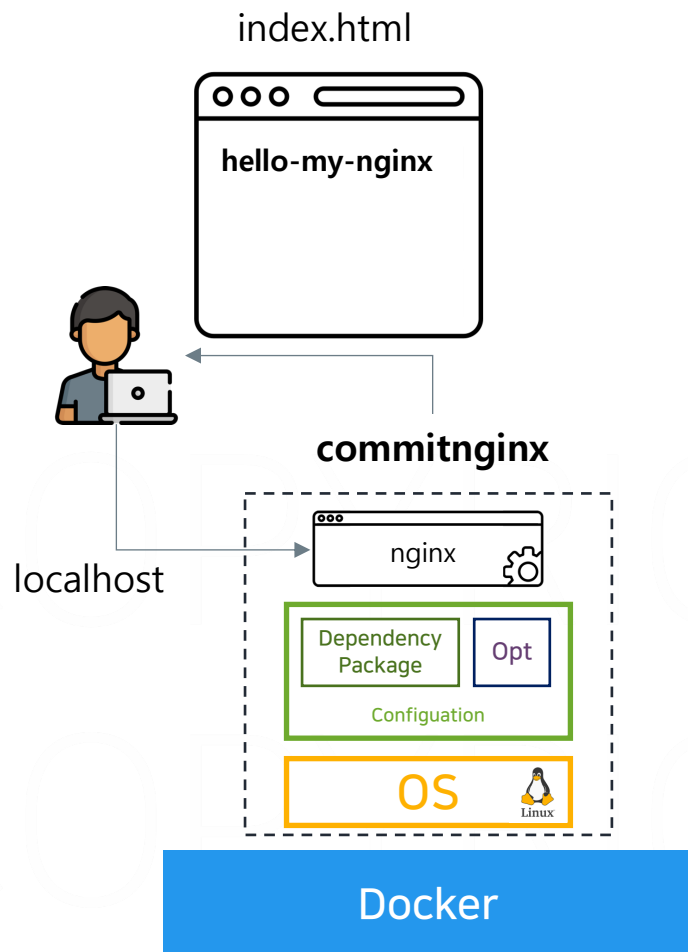
Caching (캐싱) : 레이어를 캐시하여, 이미 **빌드된 레이어를 재사용**할 수 있습니다. 이는 이미지 빌드 시간을 크게 줄여주며, 같은 레이어를 사용하는 여러 이미지에서 효율적으로 작동합니다.

PART4. 이미지 빌드

이미지 커밋

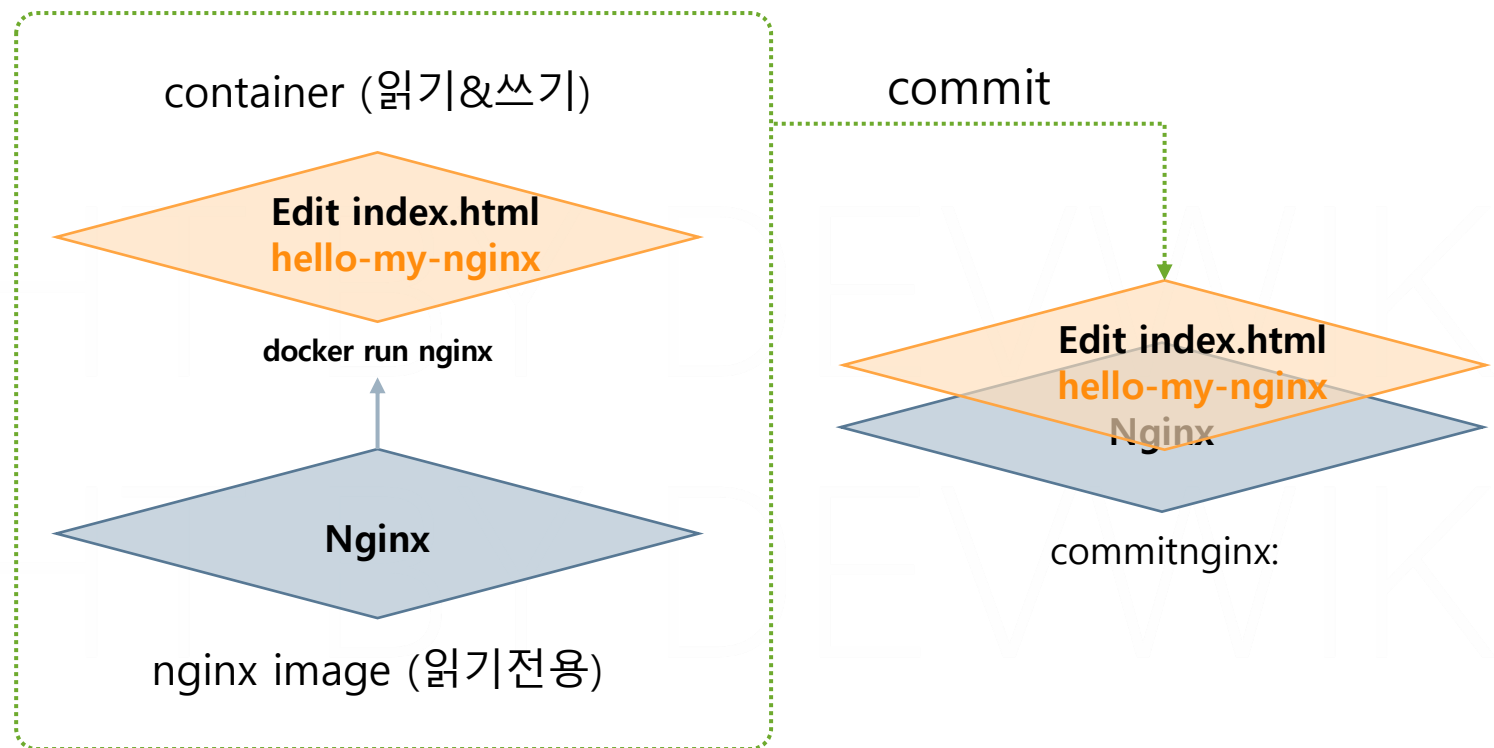
- 커밋: 현재 컨테이너의 상태를 이미지로 저장
- 빌드: Dockerfile을 통해 이미지를 저장





이미지 커밋

1. Nginx 이미지를 컨테이너로 실행
2. 컨테이너의 내부 파일 변경
3. Commit 으로 새로운 이미지로 저장



```
docker run -it --name 컨테이너명 이미지명 bin/bash
```

컨테이너 실행과 동시에 터미널 접속

```
docker commit -m 커밋명 실행중인컨테이너명 생성할이미지명
```

실행 중인 컨테이너를 이미지로 생성

nginx 컨테이너 실행 및 접속(Shell1)

1. nginx 컨테이너 실행

```
docker run -it --name officialNginx nginx bin/bash
```

[Shell2로 이동\(다음 페이지\) ==>](#)

3. index.html 파일 수정

```
cat /usr/share/nginx/html/index.html
```

```
echo hello-my-nginx > /usr/share/nginx/html/index.html
```

4. 수정한 index.html 파일 확인

```
cat /usr/share/nginx/html/index.html
```

[Shell2로 이동\(다음 페이지\) ==>](#)

이미지 커밋 (Shell2)

2. 실행된 nginx 컨테이너 확인

```
docker ps
```

<==== Shell1로 이동(이전 페이지)

5. officialNginx 컨테이너 커밋을 통해 (개인레지스트리명)/commitnginx:1.0 이미지 생성

```
docker commit -m "edited index.html by devwiki" -c 'CMD ["nginx", "-g", "daemon off;"]' officialNginx (개인레지스트리명)/commitnginx
```

6. 생성된 이미지 확인

```
docker image ls (개인레지스트리명)/commitnginx
```

7. 생성된 이미지의 히스토리 확인

```
docker image history (개인레지스트리명)/commitnginx
```

8. 커밋한 이미지로 컨테이너 실행

```
docker run -d -p 80:80 --name my-nginx (개인레지스트리명)/commitnginx
```

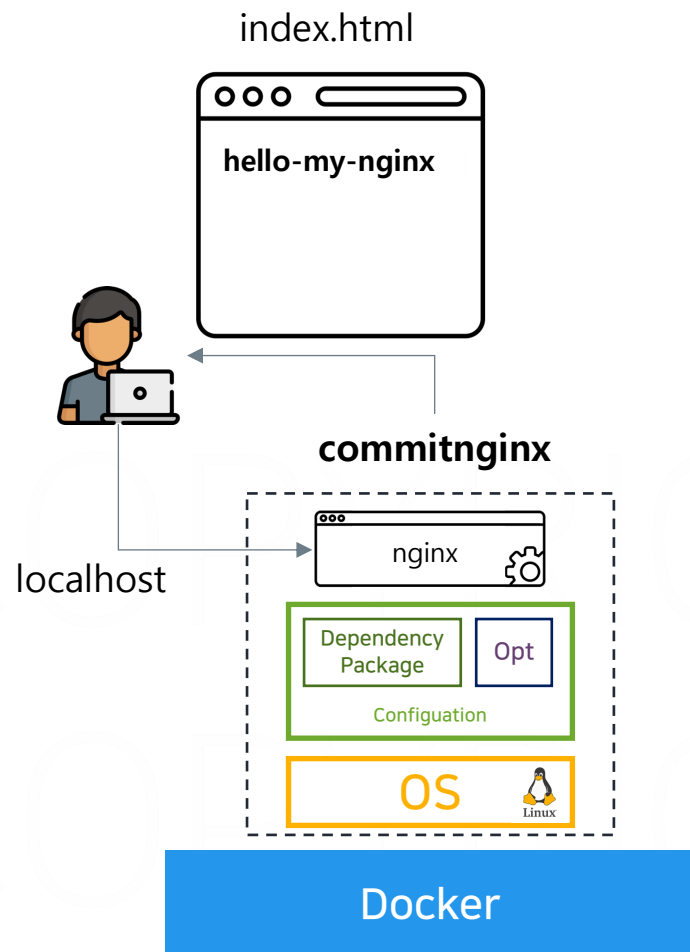
9. localhost 접속 확인 후 컨테이너 삭제

```
docker rm -f officialNginx my-nginx
```

10. 개인 레지스트리에 이미지 Push

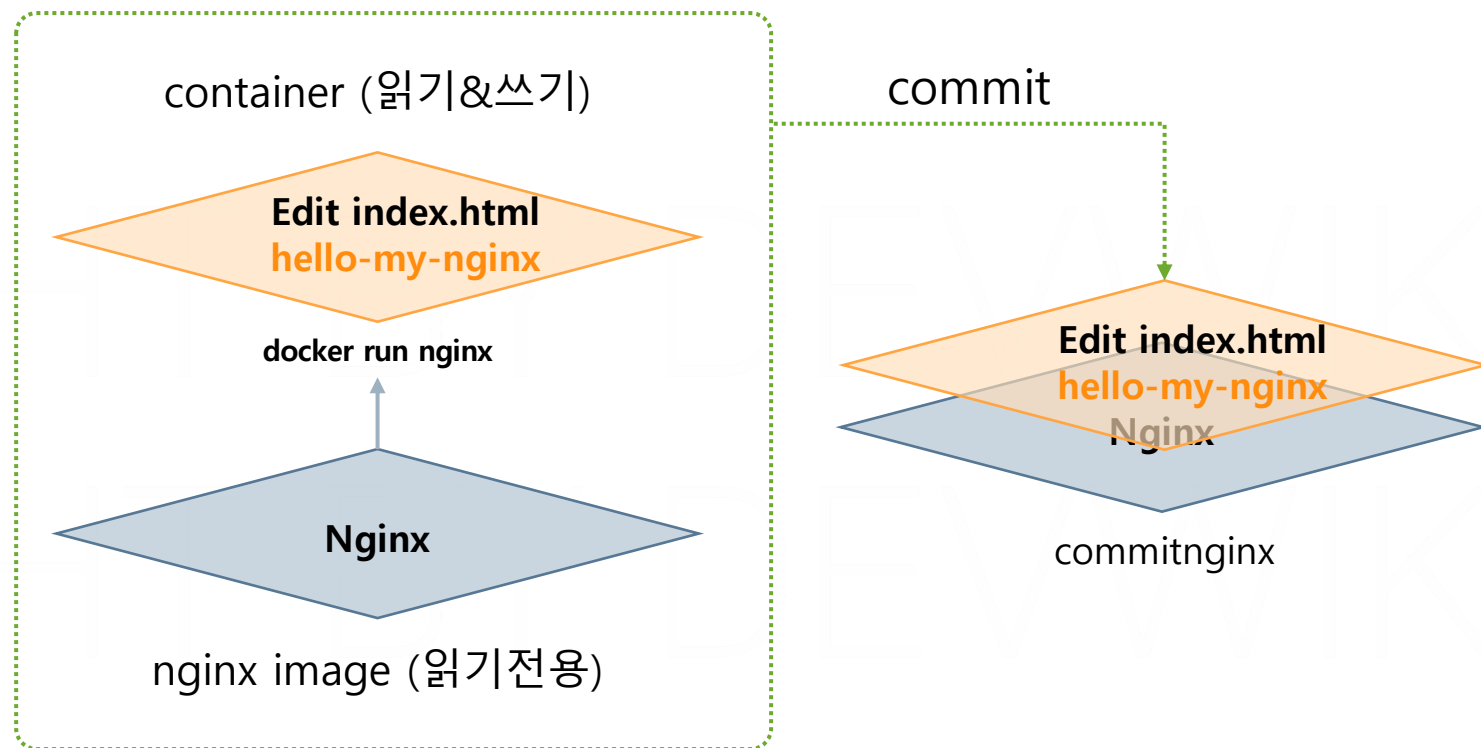
```
docker push (개인레지스트리명)/commitnginx
```

====> 실습 종료



이미지 커밋

1. Nginx 이미지를 컨테이너로 실행
2. 컨테이너의 내부 파일 변경
3. Commit 으로 새로운 이미지로 저장

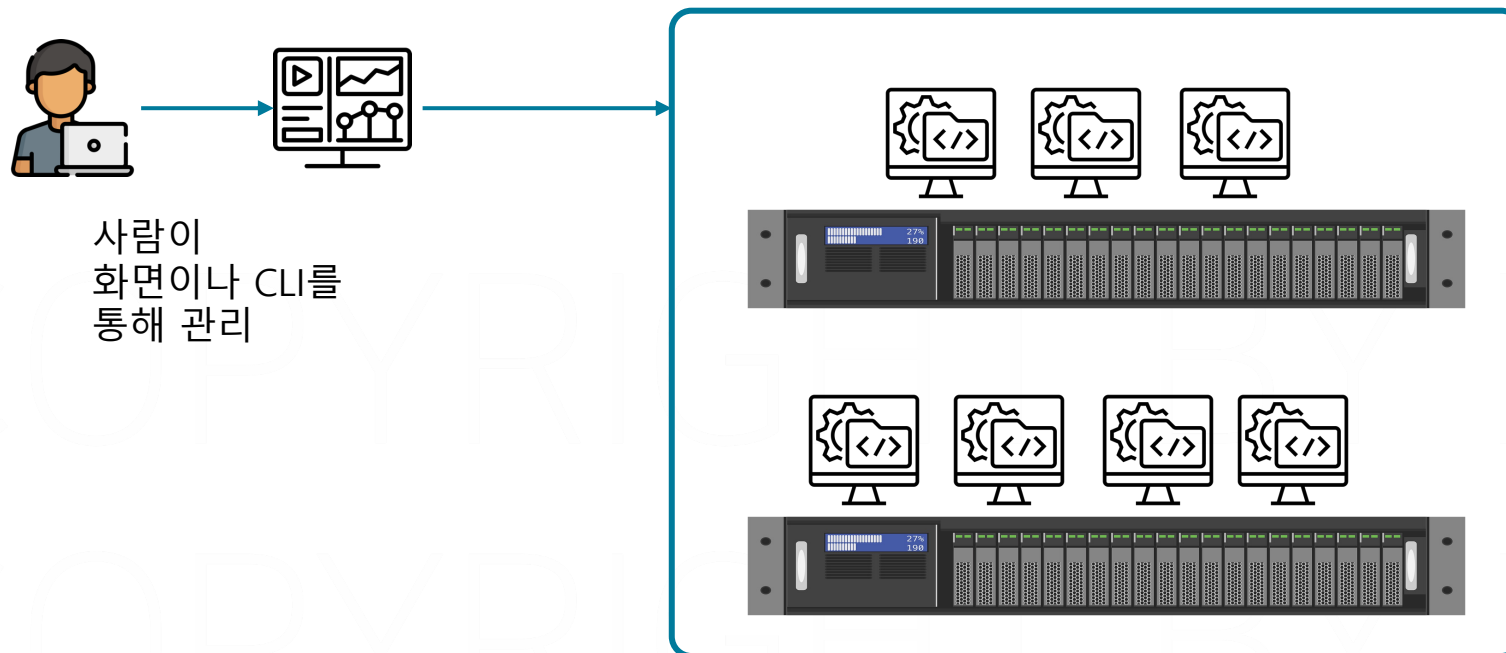


PART4. 이미지 빌드

이미지 빌드

- IaC(Infrastructure as Code): 인프라 상태를 코드로 관리

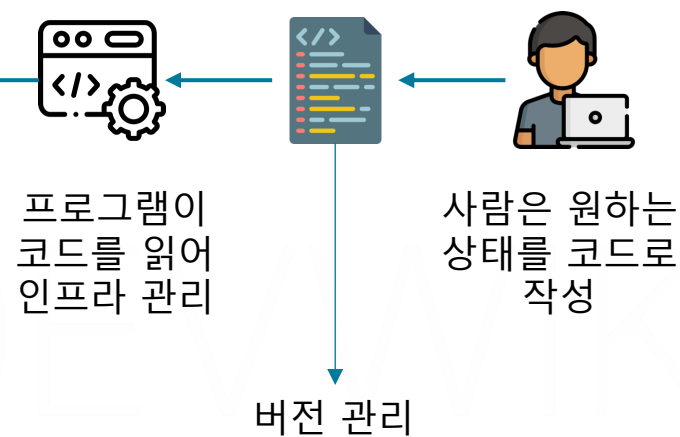
기존 방식



사람이
화면이나 CLI를
통해 관리

사내 데이터 센터

IaC 방식

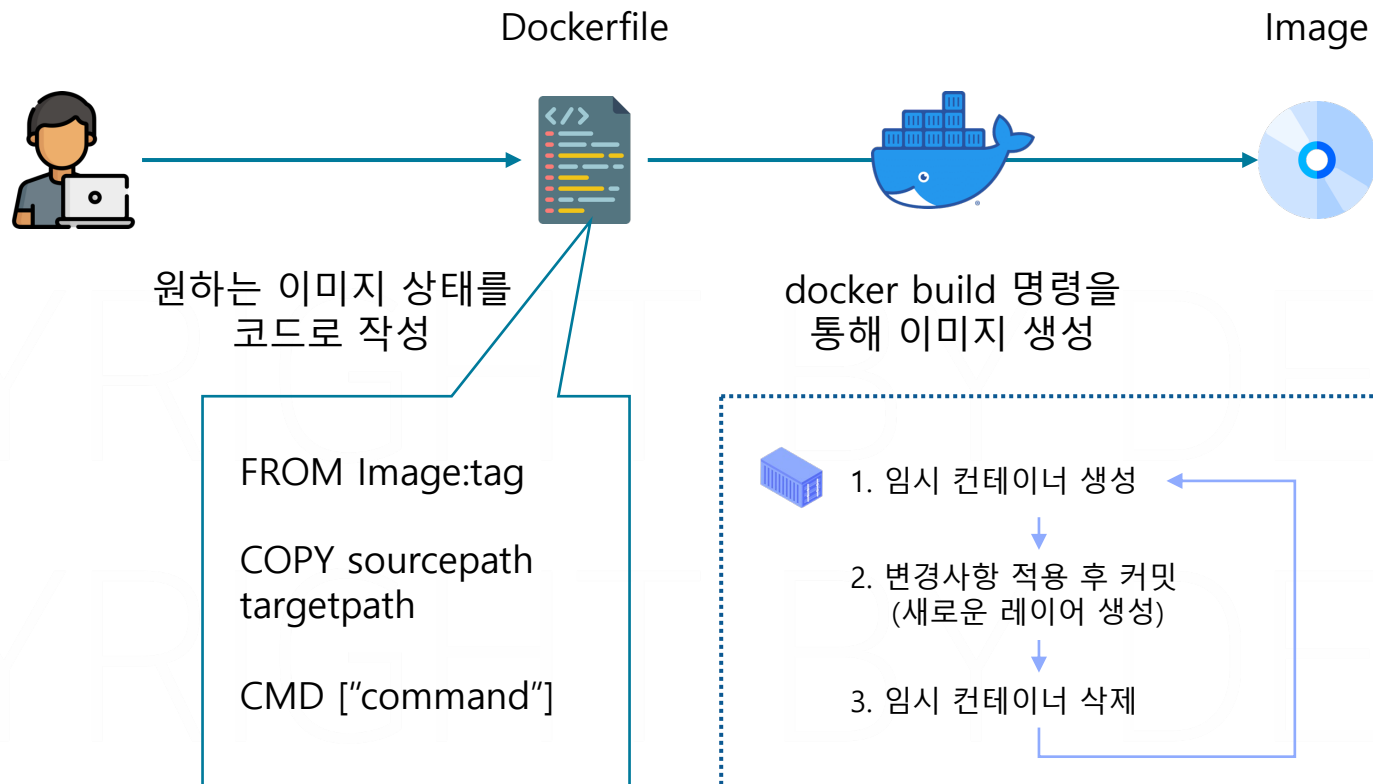


프로그램이
코드를 읽어
인프라 관리

사람은 원하는
상태를 코드로
작성

버전 관리

- Dockerfile: 이미지를 만드는 단계를 기재한 명세서



`docker build -t 이미지명 Dockerfile` 경로

도커파일을 통해 이미지 빌드

FROM 이미지명

베이스 이미지를 지정

COPY 파일경로 복사할경로

파일을 레이어에 복사

CMD ["명령어"]

컨테이너 실행 시 명령어 지정

이미지 Build 및 Push

1. easydocker/ 폴더 이동, 소스코드 다운로드

```
git clone https://github.com/daintree-henry/build.git
```

```
git switch 00-init ----- 실습 파일을 직접 작성할 경우
```

```
git switch 01-dockerfile ----- 완성된 파일로 실습하고 싶을 경우
```

2. 01.buildnginx이동 후 index.html파일 수정

```
cd 01.buildnginx
```

3. buildnginx:1.0.0 이미지 빌드(buildnginx 폴더 내에서 명령어 실행)

```
docker build -t (레파지토리계정)/buildnginx .
```

3. 빌드한 컨테이너 실행

```
docker run -d -p 80:80 --name build-nginx (개인레지스트리명)/buildnginx
```

4. localhost:8080 접속 확인 후 컨테이너 삭제

```
docker rm -f build-nginx
```

5. 개인 레지스트리에 이미지 Push

```
docker push (개인레지스트리명)/buildnginx
```

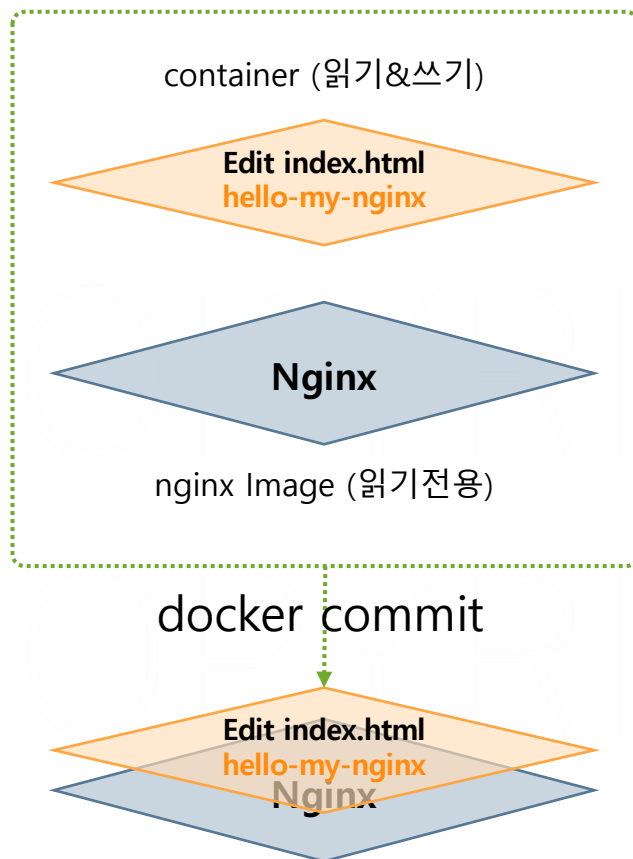
easydocker/build/01.buildnginx/Dockerfile

```
FROM nginx:1.23
```

```
COPY index.html /usr/share/nginx/html/index.html
```

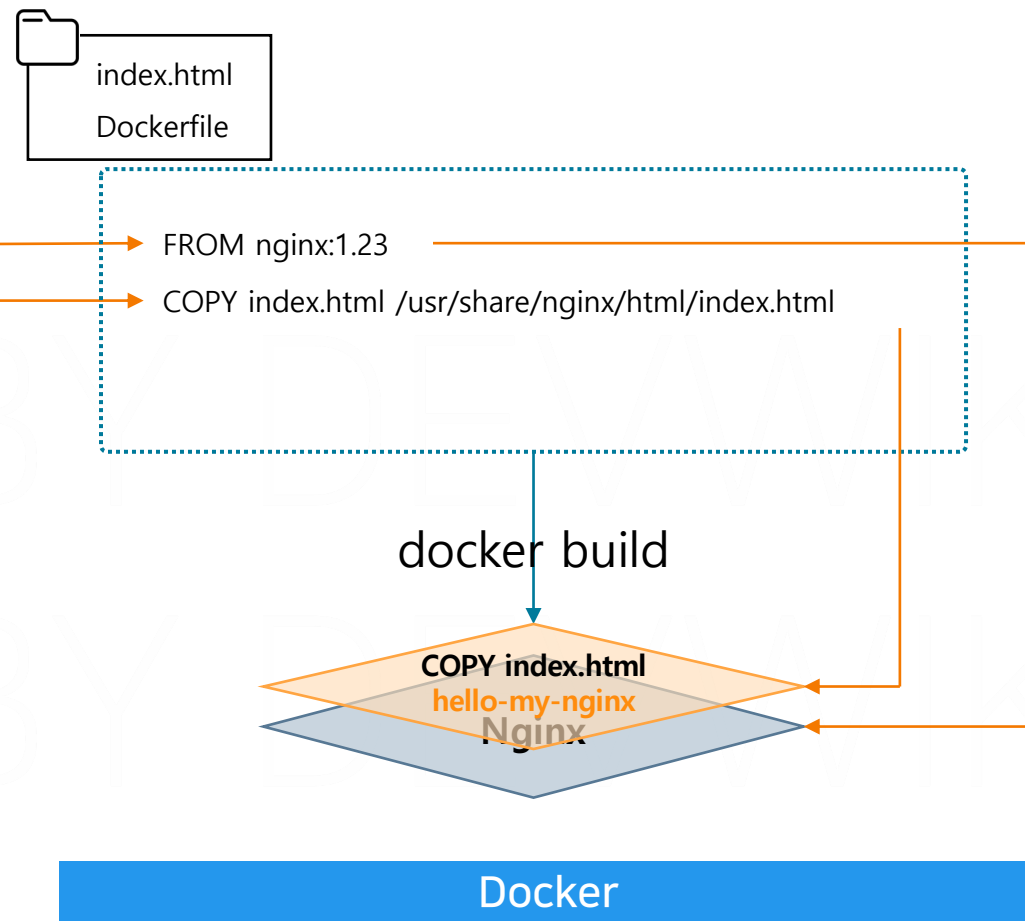
```
CMD ["nginx", "-g", "daemon off;"] ----- CMD 명령어 지정(nginx -g daemon off;)
```

Commit (사람이 이미지 생성)



1. Nginx 이미지를 컨테이너로 실행
2. 컨테이너의 내부 파일 변경
3. Commit 으로 새로운 이미지로 저장

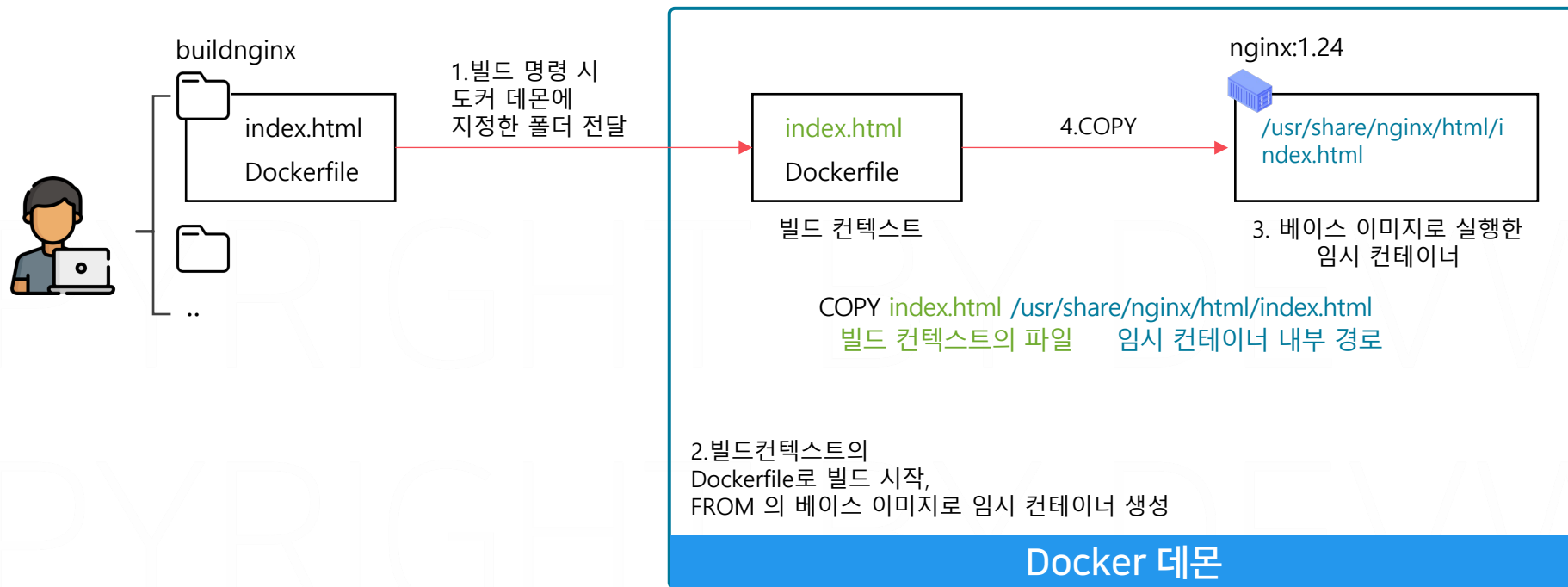
Build (도커 데몬이 이미지 생성)



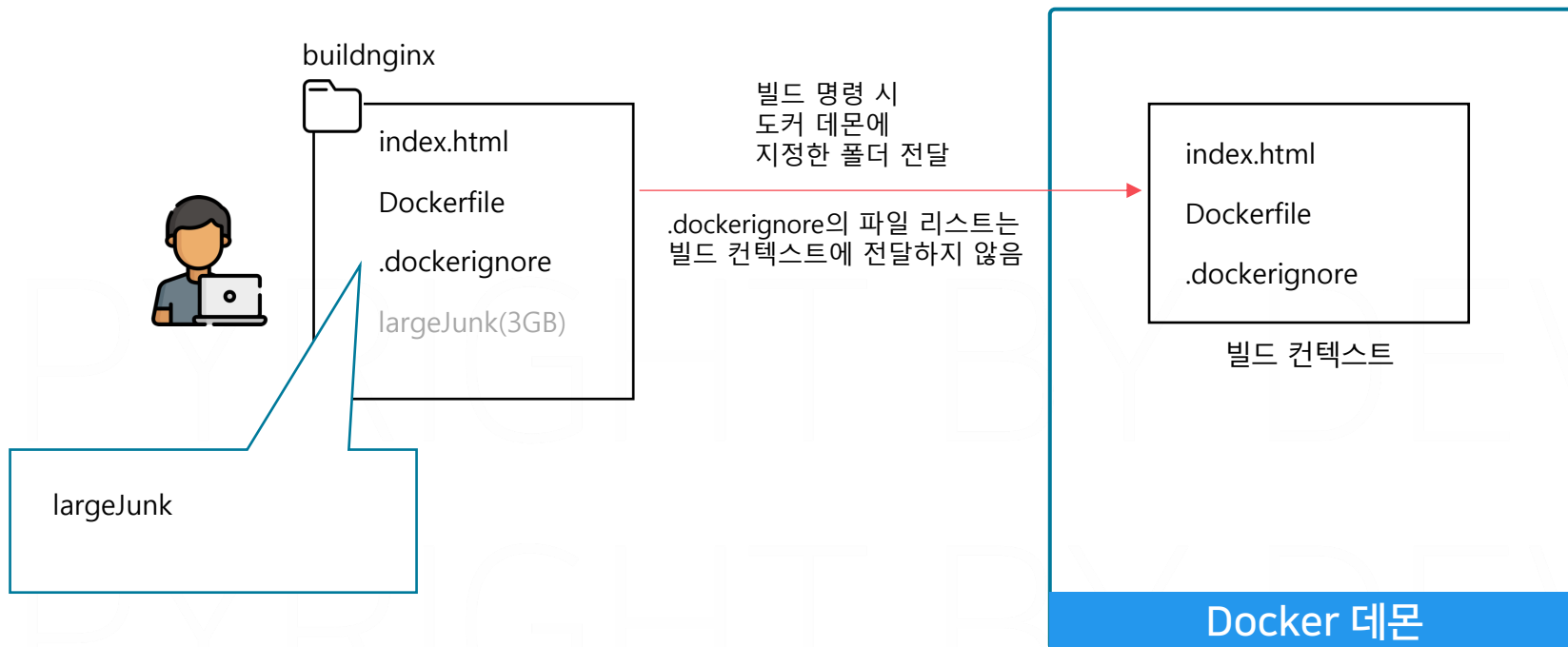
PART4. 이미지 빌드

빌드 컨텍스트

```
docker build -t my-image-name .
```



docker build -t my-image-name .



.dockerignore 테스트

1. easydocker/part4/02.buildcontext 폴더 이동, 파일 내용 확인 후 이미지 빌드

```
cd ../02.buildcontext
```

2. 빌드 시도 및 실패한 결과 확인, 컨텍스트에 largeJunk 가 없어 실패

```
docker build -t buildcontext:ignorejunk .
```

easydocker/build/02.buildcontext/largeJunk.txt

It is the large file

easydocker/build/02.buildcontext/Dockerfile

FROM nginx:1.23 ----- 1.23 버전의 nginx를 베이스 이미지로 사용

COPY largeJunk.txt /usr/share/nginx/html/index.html ----- 이미지 내 index.html 파일을 수정

CMD ["nginx", "-g", "daemon off;"] ----- CMD 명령어 지정

easydocker/build/02.buildcontext/.dockerignore

largeJunk.txt

PART4. 이미지 빌드

Dockerfile 지시어

← → ↻ ⓘ localhost:8080

Env Color Application

사용자님, 환영합니다.

현재 ENV 값으로 적용된 환경 변수는 red 입니다.

COLOR=red

← → ↻ ⓘ localhost:8081/henry

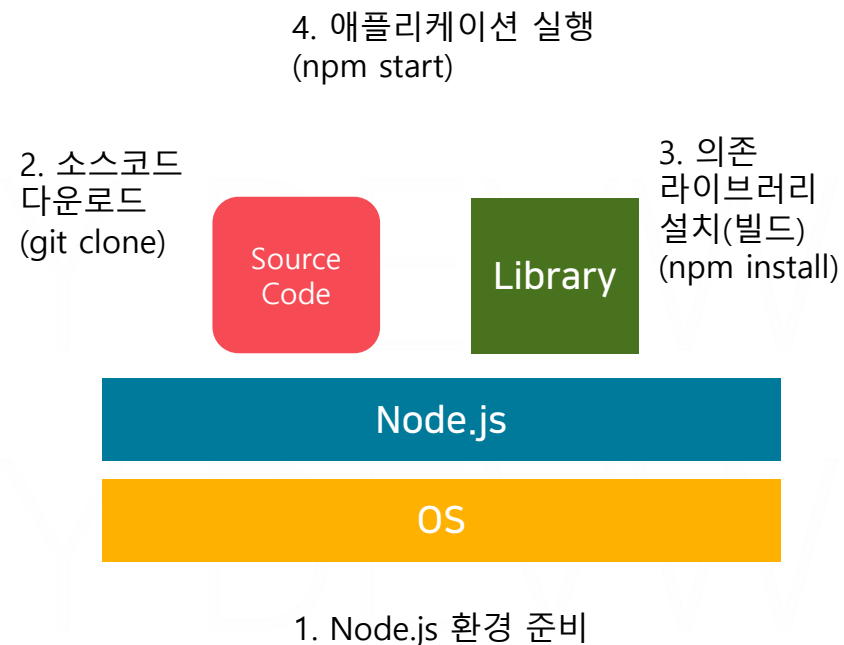
Env Color Application

henry님, 환영합니다.

현재 ENV 값으로 적용된 환경 변수는 blue 입니다.

COLOR=blue
URL= /{이름}

Node.js 애플리케이션 구성 및 실행 과정



이미지 빌드

애플리케이션 빌드

소스코드를 실행 가능한
프로그램으로 빌드

Source
Code

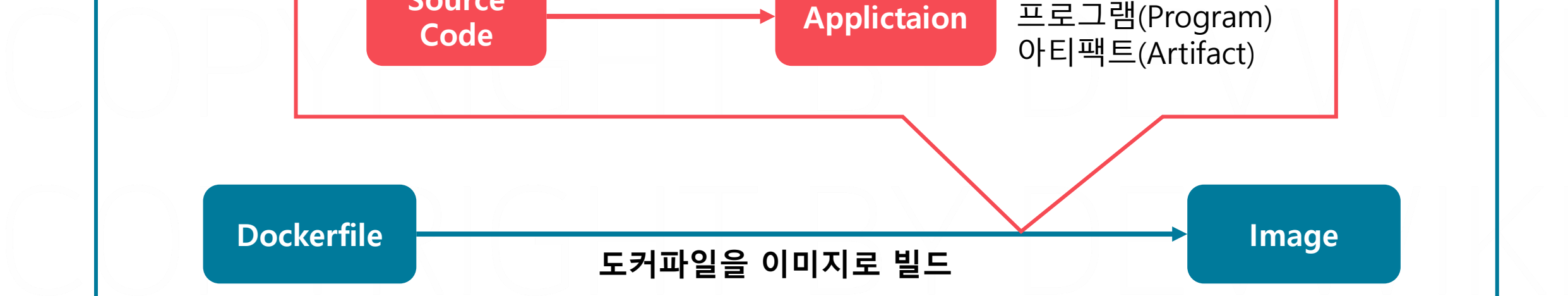
Applictaiion

애플리케이션
프로그램(Program)
아티팩트(Artifact)

Dockerfile

도커파일을 이미지로 빌드

Image



src/app.js

```
app.get('/', (req, res) => {
  // 요청을 받을 URI
  res.render('index', { color, username });
});
```

▼ 시스템의 COLOR 환경 변수 읽어옴

▲ index.ejs 파일 응답, color, username 전달

src/views/index.ejs

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
  initial-scale=1.0">
  <title>Simple Node.js Web App</title>
  <style>
    body {
      color: <%= color %>;
    }
  </style>
</head>
<body>
  <h1>Env Color Application</h1>
  <p><%= username %>님, 환영합니다.</p>
  <p>현재 ENV 값으로 적용된 환경 변수는 <%= color %>
  입니다.</p>
</body>
```

▲ color값을 CSS 컬러값으로 지정

페이지 내 사용자명을 username으로 지정

COPYRIGHT

FROM 이미지명

베이스 이미지를 지정

COPY 빌드컨텍스트경로 레이어경로

빌드 컨텍스트의 파일을 레이어에 복사

(cp) (새로운 레이어 추가)

RUN 명령어

명령어 실행

(새로운 레이어 추가)

CMD ["명령어"]

컨테이너 실행 시 명령어 지정

docker build -f 도커파일명 -t 이미지명 Dockerfile경로

도커파일명이 Dockerfile이 아닌 경우 별도 지정

Build 테스트

1. 03.envcolorapp 실습 폴더로 이동

```
cd ../03.envcolorapp
```

2. 이미지 빌드

```
docker build -f Dockerfile-basic -t buildapp:basic .
```

easydocker/build/03.envcolorapp/Dockerfile-basic

FROM node:14 ----- 14 버전의 node를 베이스 이미지로 사용

COPY ./ / ----- 빌드 컨텍스트의 전체 파일을 / 으로 이동

RUN npm install ----- 애플리케이션 의존 라이브러리 설치

CMD ["npm", "start"] ----- 컨테이너 실행 시 애플리케이션 실행 명령

WORKDIR 폴더명

작업 디렉토리를 지정 (cd)
(새로운 레이어 추가)

USER 유저명

명령을 실행할 사용자 변경 (su)
(새로운 레이어 추가)

EXPOSE 포트번호

컨테이너가 사용할 포트를 명시

Build 테스트

1. 이미지 빌드

```
docker build -f Dockerfile-meta -t buildapp:meta .
```

2. 컨테이너 실행 및 테스트 (localhost:3000)

```
docker run -d -p 3000:3000 --name buildapp-meta buildapp:meta
```

easydocker/build/03.envcolorapp/Dockerfile-meta

```
FROM node:14 ----- 14 버전의 node를 베이스 이미지로 사용
WORKDIR /app ----- 작업 디렉토리를 app으로 이동
COPY . . ----- 빌드 컨텍스트의 전체 파일을 /app 으로 이동
RUN npm install ----- 애플리케이션 의존 라이브러리 설치
USER node ----- 작업 사용자를 node로 변경
EXPOSE 3000 ----- 애플리케이션이 사용할 포트 지정
CMD ["npm", "start"]----- 컨테이너 실행 시 애플리케이션 실행 명령
```

ARG 변수명 변수값

이미지 빌드 시점의 환경 변수 설정

`docker build --build-arg 변수명=변수값` 으로 덮어쓰기 가능

ENV 변수명 변수값

이미지 빌드 및 컨테이너 실행 시점의 환경 변수 설정
(새로운 레이어 추가)

`docker run -e 변수명=변수값` 으로 덮어쓰기 가능

easydocker/build/03.envcolorapp/Dockerfile-arg

```
FROM node:14
WORKDIR /app
COPY ./ /app/
RUN npm install
ARG COLOR=red
USER node
EXPOSE 3000
CMD ["npm", "start"]
```

easydocker/build/03.envcolorapp/Dockerfile-env

```
FROM node:14
WORKDIR /app
COPY ./ /app/
RUN npm install
ENV COLOR=red
USER node
EXPOSE 3000
CMD ["npm", "start"]
```

Build 테스트

1. Dockerfile 수정, ARG 이미지 빌드

```
docker build -f Dockerfile-arg -t buildapp:arg .
```

2. Dockerfile 수정, ENV 이미지 빌드

```
docker build -f Dockerfile-env -t buildapp:env .
```

3. 컨테이너 실행 및 arg, env 적용 이미지의 컬러 확인

```
docker run -d --name buildapp-arg -p 3001:3000 buildapp:arg
```

```
docker run -d --name buildapp-env -p 3002:3000 buildapp:env
```



localhost:3001

Env Color Application

사용자님, 환영합니다.

현재 ENV 값으로 적용된 환경 변수는 green 입니다.



localhost:3002

Env Color Application

사용자님, 환영합니다.

현재 ENV 값으로 적용된 환경 변수는 red 입니다.

ENTRYPOINT ["명령어"]

고정된 명령어를 지정

CMD ["명령어"]

컨테이너 실행 시 실행 명령어 지정

```
easydocker/build/03.envcolorapp/Dockerfile-entrypoint
```

```
FROM node:14
```

```
WORKDIR /app
```

```
COPY ./ /app/
```

```
RUN npm install
```

```
USER node
```

```
EXPOSE 3000
```

```
ENTRYPOINT ["npm"] ----- 컨테이너 실행 시 고정 실행 명령
```

```
CMD ["start"] ----- 컨테이너 실행 시 명령 파라미터
```

Build 테스트

1. 이미지 빌드

```
docker build -f Dockerfile-entrpoint -t buildapp:entrpoint .
```

2. 컨테이너 실행 (ENTRYPOINT: npm / CMD: list 실행)

```
docker run --name buildapp-entrpoint-list buildapp:entrpoint list
```

3. 컨테이너 실행 (ENTRYPOINT: npm / CMD: /bin/bash 실행)

```
docker run -it --name buildapp-entrpoint-bash buildapp:entrpoint /bin/bash
```

4. 실습 컨테이너 삭제

```
docker rm -f buildapp-meta buildapp-arg buildapp-env buildapp-entrpoint-list buildapp-entrpoint-bash
```

PART4. 이미지 빌드

멀티 스테이지 빌드

백엔드 Spring Boot Application 이미지 구성

5. 애플리케이션 실행

```
java -jar app.jar
```

4. 의존성 라이브러리 설치 및 빌드

```
mvn clean package
```

3. 소스코드 다운로드

```
git clone ...
```

2. 빌드 도구 설치

```
mvn
```

1. OS구성 및 Java Runtime 설치

Java Runtime

OS



app.jar

JavaHelloApp



localhost:8080/hello

Hello, World!

Multi-Stage 백엔드 Spring Boot Application 이미지 구성

CMD ["java", "-jar", "app.jar"]

FROM openjdk:11-jre-slim

실행 스테이지

5. 애플리케이션 실행

java -jar app.jar



4. 의존성 라이브러리 설치 및 빌드

mvn clean package

3. 소스코드 다운로드

git clone ...

2. 빌드 도구 설치

mvn

1. OS구성 및 Java Runtime 설치

Java Runtime

OS

RUN mvn clean package

COPY pom.xml .

COPY src ./src

FROM maven:3.6 AS build

빌드 스테이지

Build 테스트

```
04.javahelloapp 실습 폴더로 이동  
cd ../04.javahelloapp
```

```
easydocker/build/04.javahelloapp/Dockerfile.singlestage
```

```
FROM maven:3.6-jdk-11 ----- JAVA 애플리케이션을 빌드, 실행 가능한 베이스 이미지
```

```
WORKDIR /app
```

```
COPY pom.xml . ----- 의존 라이브러리 정보가 저장된 pom.xml파일 /app 으로 복사
```

```
COPY src ./src ----- 애플리케이션 소스파일 /app 으로 복사
```

```
RUN mvn clean package ----- 애플리케이션 빌드
```

```
RUN cp /app/target/*.jar ./app.jar ----- 빌드된 아티팩트 복사
```

```
EXPOSE 8080
```

```
CMD ["java", "-jar", "app.jar"] ----- 컨테이너 실행시 아티팩트 실행
```

```
easydocker/build/04.javahelloapp/Dockerfile
```

```
# 첫번째 단계: 빌드 환경 설정
```

```
FROM maven:3.6 AS build ----- JAVA 애플리케이션을 빌드하기 위해 maven 베이스 이미지 사용
```

```
WORKDIR /app
```

```
# 소스코드 복사
```

```
COPY pom.xml .
```

```
COPY src ./src
```

```
# 애플리케이션 빌드
```

```
RUN mvn clean package ----- JAVA 애플리케이션을 빌드
```

```
# 두번째 단계: 실행 환경 설정
```

```
FROM openjdk:11-jre-slim ----- JAVA 애플리케이션을 실행하기 위해 openjdk 베이스 이미지 사용
```

```
WORKDIR /app
```

```
# 빌드 단계에서 생성된 JAR 파일 복사
```

```
COPY --from=build /app/target/*.jar ./app.jar ----- build 에서 실행 이미지로 파일 복사
```

```
EXPOSE 8080
```

```
CMD ["java", "-jar", "app.jar"]
```

Build 테스트

1. 단일 스테이지 이미지 빌드 및 조회

```
docker build -f Dockerfile.singlestage -t javaappsingle .
```

```
docker image ls javaappsingle
```

2. 멀티 스테이지 이미지 빌드 및 조회

```
docker build -f Dockerfile.multistage -t javaappmulti .
```

```
docker image ls javaappmulti
```

✓ SIMPLEJAVAHELLOAPP

✓ src\main\java\com\example\demo

J DemoApplication.java

 Dockerfile.multistage Dockerfile.singlestage pom.xml

\$ script.sh

[javaappmulti](#)133bea8ed578 

latest

Unused

4 minutes ago

232.95 MB

[javaappsingle](#)0d8996025a55 

latest

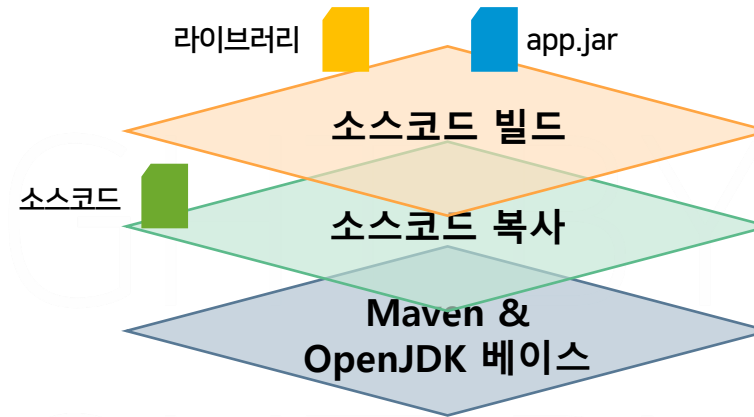
Unused

11 minutes ag

730.53 MB

Dockerfile.singlestage

730.53 MB



FROM maven:3.6-jdk-11

- 실행 단계에 불필요한 소스코드, 라이브러리, 실행파일 등 포함

Dockerfile.multistage

